

Combinatorial and Dynamic Multi-Agent Path Finding: Exact and Heuristic Solvers with Scalable Visualization

Stefan Edelkamp

Department of Theoretical Computer Science and Mathematical Logic
Faculty of Mathematics and Physics, Charles University, Prague, Czech Republic

Abstract

We study Combinatorial Multi-Agent Path Finding (CMAPF) in discrete grid-based and continuous roadmap-based settings. Given k robots and n target waypoints, the task is to partition waypoints among robots and find open Hamiltonian paths optimizing a linear combination of makespan and sum-of-costs.

In the discrete variant, robots move on grid cells and collisions are resolved by swapping precomputed BFS trees. Under the mild assumption that only opposing edge and node conflicts occur on the overlay of shortest paths, CMAPF and AMAPF are optimal with respect to the underlying waypoint-to-robot assignment. Seven solvers are compared on 20 maps with up to over 8,000 agents; a free-cell memory optimization reduces RAM tenfold, and delta-encoded visualization achieves $330\times$ compression; combined with binary frame encoding, the full benchmark suite of 1,864 runs totals under 1 GB.

The continuous variant replaces the grid with a roadmap graph generated by adaptive distance-transform sampling. Robots move along Bézier-smoothed shortest paths with C^1 continuity at curvature-adaptive speed. Collisions on graph edges are resolved by identity exchange at analytically precomputed meeting points, proven to be the exact elastic solution for equal-mass point particles. Dijkstra’s algorithm replaces BFS on the weighted roadmap, and if the mTSP is solved exactly on the graph, the continuous CMAPF solution is optimal with respect to that graph. The entire pipeline runs smoothly on a single core of a 16 GB laptop.

Introduction

Multi-Agent Path Finding (MAPF) is a fundamental problem in AI and robotics: given a set of agents with start and goal positions in a shared environment, find collision-free paths that minimize a global cost objective. In the *combinatorial* variant (CMAPF), the number of target waypoints n is typically much larger than the number of robots k , so waypoints must be partitioned among robots and sequenced into tours. This reduces to a multi-depot open Multiple Traveling Salesmen Problem (mTSP) on the waypoint distance graph. Unlike standard mTSP, which typically minimizes makespan alone, our formulation optimizes an

arbitrary user-defined linear combination of makespan and sum-of-costs (Section), with the additional requirement that the resulting tours be executable without collisions.

We study CMAPF in two settings: discrete grid-based maps from the Moving AI benchmark set [Sturtevant(2012)], and continuous roadmap-based environments generated by adaptive distance-transform sampling (also derived from the Moving AI benchmark set [Sturtevant(2012)]). In both settings, collisions are resolved deterministically: by swapping precomputed BFS trees on grids, and by identity exchange at analytically precomputed meeting points on roadmap edges.

Our main contributions are:

- A unified mTSP formulation for discrete and continuous CMAPF, optimizing a linear combination of makespan and sum-of-costs (Section).
- Seven solvers—SA, k -medoids, NRPA, LKH-3, branch-and-bound, AMAPF, and DMAPF—evaluated on 20 Moving AI maps with up to over 8,000 agents (Section).
- An optimality result: under the mild assumption that only opposing edge and node conflicts occur on the overlay of assigned shortest paths, the collision resolution in CMAPF and AMAPF is optimal with respect to the waypoint-to-robot assignment.
- A free-cell memory optimization that reduces BFS storage from $n \times W \times H$ to $n \times F$, enabling experiments at $n = 6,820$ that were previously infeasible on commodity hardware (Section).
- Delta encoding and binary frame compression achieving $330\times$ and $19\times$ reductions respectively, totaling under 1 GB for the full benchmark suite (Section).
- A continuous CMAPF extension with adaptive roadmap generation that handles all maps in the Moving AI benchmark set, and provably correct, framerate-independent collision detection via analytical precomputation (Section).
- Bézier curve path smoothing producing C^1 -continuous trajectories from piecewise-linear graph edges, with arc-length reparameterization for constant-speed traversal (Section).

- A physical derivation showing that the identity-exchange collision resolution is the exact solution for elastic collisions of equal-mass bodies, with generalized bounding for finite-radius robots (Section).
- Dijkstra’s algorithm replacing BFS on weighted roadmap graphs, enabling exact all-pairs shortest paths and guaranteeing optimal CMAPF solutions when the mTSP is solved exactly (Section).
- The entire pipeline—solving, simulation, and visualization—runs on a single core of a 16 GB laptop.

The paper is organized as follows. Section defines the discrete CMAPF problem and the objective function. The algorithmic framework, which applies to both discrete and continuous settings, is presented in Section . Section covers the experimental setup, results, and visualization. Section extends the formulation to continuous roadmap graphs and describes the collision precomputation theory. Section concludes.

Problem Definition (Discrete)

Given an undirected grid map $\mathcal{M}$ of size $W \times H$, a set of k robots $R = \{r_1, \dots, r_k\}$ with distinct start positions $s_1, \dots, s_k$, and a set of n target waypoints $W = \{w_1, \dots, w_n\}$ (where the start positions are also waypoints), find:

1. A partition of W into k disjoint subsets $W_1, \dots, W_k$, with $s_r \in W_r$,
2. For each robot r , an open Hamiltonian path starting at s_r and visiting all waypoints in W_r exactly once, such that the objective function

$$f = \alpha \cdot M + \beta \cdot \sum_{r=1}^k C_r \quad (1)$$

is minimized, where $M = \max_r C_r$ is the makespan and C_r is the total cost of robot r ’s tour. The weights $\alpha, \beta \geq 0$ control the trade-off between the longest individual tour and the total system effort. Throughout this paper we instantiate $\alpha = 1$ and $\beta = 1/k$, yielding $f = M + \frac{1}{k} \sum_{r=1}^k C_r$, which C_r is the total travel cost of robot r ’s path.

Edge costs $d(i, j)$ are the shortest-path distances on $\mathcal{M}$ precomputed via BFS from every waypoint. The problem is NP-hard (reduces to TSP for $k = 1$).

Algorithms

All solvers share a common preprocessing phase: BFS from each waypoint computes all-pairs shortest-path distances $d(i, j)$ on the map. In the discrete case the map is a grid; in the continuous case it is a weighted roadmap graph (Section). The distance matrix d is provided to all solvers. The solvers described in the remainder of this section—SA, k -medoids, NRPA, LKH-3, and branch-and-bound—operate solely on the distance matrix and are therefore agnostic to the underlying map representation; they apply identically to both

the discrete grid-based and continuous roadmap-based settings.

Anytime behavior. NRPA and SA are *anytime* algorithms: they continuously improve their best solution and can be stopped at any point, returning the best result found so far. An internal iteration counter is adapted to an external time budget: when the allotted wall-clock time is nearly exhausted, the solver terminates exploration and reports the incumbent best solution. Consequently, solution quality improves monotonically with the time limit; longer runs yield progressively better scores. In our benchmarks we use a 300 s timeout per solver per instance, but the same code path supports arbitrary time limits.

Simulated Annealing (SA)

SA maintains a complete solution (partition + tours) and improves it through stochastic local search with three move types.

Algorithm 1 Simulated Annealing for mTSP

```

1: Parameters:  $T_0 = 50$ ,  $T_{\min} = 0.05$ ,  $\alpha = 0.98$ , 600 moves/temperature
2: Build Voronoi initial partition (nearest start by  $d$ )
3: Nearest-neighbor + 2-OPT per cluster for initial tours
4:  $best \leftarrow$  current solution
5: while  $T > T_{\min}$  do
6:   for  $m = 1$  to 600 do
7:      $p \leftarrow \text{random}[0, 1]$ 
8:     if  $p < 0.5$  then
9:       2-OPT on random robot’s tour (reverse segment)
10:    else if  $p < 0.95$  then
11:      Relocate: move one waypoint between robots
12:    else
13:      Swap: exchange two waypoints between robots
14:    end if
15:     $\Delta \leftarrow$  change in objective  $f$ 
16:    if  $\Delta < 0$  or  $\text{random}() < e^{-\Delta/T}$  then
17:      Accept move; update best if improved
18:    end if
19:  end for
20:   $T \leftarrow T \cdot \alpha$ 
21: end while
22: return best solution

```

SA is anytime: the temperature schedule can be interrupted at any point, returning the best solution found so far. In our implementation, an internal iteration counter is calibrated to the external time limit so that the cooling schedule completes just before the deadline.

Balanced k -Medoids + 2-OPT (KMEDOIDS)

k -medoids clusters waypoints using shortest-path distances with load-balancing penalties, then routes each cluster via 2-OPT.

Algorithm 2 Balanced k -Medoids + 2-OPT

```

1: Parameters:  $\alpha = 0.45$  (size balance),  $\beta = 0.65$  (travel balance)
2: Initialize medoids  $m_r \leftarrow s_r$ , assign each  $s_r$  to cluster  $r$ 
3: for iter = 1 to 1000 do
4:   Compute cluster sizes  $|W_r|$ , avg size  $\bar{s} = n/k$ 
5:   Compute travel sum  $T_r = \sum_{w \in W_r} d(w, m_r)$ , avg travel  $\bar{t}$ 
6:   for each non-start waypoint  $w$  do
7:     Assign  $w$  to robot  $r$  minimizing
8:      $d(w, m_r) \cdot (1 + \alpha \frac{|W_r| - \bar{s}}{\bar{s}}) \cdot (1 + \beta \frac{T_r - \bar{t}}{\bar{t}})$ 
9:   end for
10:  if no change then
11:    break
12:  end if
13:  Update medoid of each cluster:
14:   $m_r \leftarrow \arg \min_{c \in W_r} \sum_{w \in W_r \setminus \{c\}} d(c, w)$ 
15: end for
16: for each cluster  $r$  do
17:   Build nearest-neighbor tour starting at  $s_r$ 
18:   Apply 2-OPT local search on the open path
19: end for
20: return concatenated tours, objective  $f$ 

```

Nested Rollout Policy Adaptation (NRPA)

NRPA learns a stochastic policy $\pi(i, j)$ over waypoint transitions through nested Monte-Carlo rollouts. At each level ℓ , it performs I rollouts using the current policy, selecting successors via Gibbs sampling with the policy weights. The best rollout sequence adapts the policy of level $\ell - 1$.

NRPA explores the full combined assignment-routing space without decomposing into clustering + routing. The learned policy captures which waypoint-to-waypoint transitions produce good solutions. Nested levels allow deeper exploration of promising regions. The anytime nature is intrinsic: the number of rollout iterations I is scaled by n to fit within the allotted time budget, and the algorithm can be stopped at any iteration, returning the best solution found up to that point.

LKH-3 Clustered Solver

The Lin–Kernighan–Helsgaun (LKH-3) [Helsgaun(2017)] heuristic is a state-of-the-art TSP and mTSP solver. We integrate it in two forms.

Clustered variant. The solver first clusters waypoints via balanced k -medoids (Sec.), then invokes

Algorithm 3 NRPA for mTSP

```

1: Parameters:  $\ell_{\max} = 2$ ,  $I = 150000/n$ , temperature  $\tau = 1/\ell$ 
2: Initialize global policy  $\pi(i, j) = \text{avg}(d(i, \cdot))/d(i, j)$ 
3: return NRPA( $\ell_{\max}$ )
4: procedure NRPA( $\ell$ )
5:   if  $\ell = 0$  then
6:     return ROLLOUT()  $\triangleright$  random policy
7:   end if
8:    $best \leftarrow (\text{score} = \infty)$ 
9:   Backup policy:  $\pi_\ell \leftarrow \pi$ 
10:  for  $t = 1$  to  $I$  do
11:     $(\text{score}, \text{tour}) \leftarrow \text{NRPA}(\ell - 1)$ 
12:    Adapt policy:  $\pi(i, j) \leftarrow \pi(i, j) + \frac{1}{\tau} \cdot \mathbb{1}[\text{edge}(i, j) \text{ in best tour}]$ 
13:    if  $\text{score} < best.\text{score}$  then
14:       $best \leftarrow (\text{score}, \text{tour})$ 
15:    end if
16:  end for
17:  Restore policy:  $\pi \leftarrow \pi_\ell$   $\triangleright$  discard adaptations
18:  return  $best$ 
19: end procedure
20: procedure ROLLOUT
21:  Build tour greedily: start at  $s_0$ , mark start positions as delimiters
22:  while not all waypoints visited do
23:    Select next unvisited waypoint  $j$  from current  $i$  via Gibbs:
24:     $P(j|i) \propto \exp(\pi(i, j)/\tau)$ 
25:    Mark  $j$  visited; if  $j$  is a start, begin new robot route
26:  end while
27:  Compute  $f = M + \frac{1}{k} \sum C_r$  from built tour
28:  return  $(f, \text{tour})$ 
29: end procedure

```

LKH-3 as a standalone TSP solver on each robot’s waypoint subset independently. For a robot with m assigned waypoints, the LKH wrapper:

1. Extracts the $m \times m$ distance sub-matrix from the global $n \times n$ matrix;
2. Calls `lkh_solve_mTSP` with 1 salesman, min-sum objective, 2000 max trials, and a 5s time limit per robot;
3. Extracts the TSP cycle cost and subtracts the longest edge to obtain the open-path cost (robots end at their last waypoint, not returning to start).

LKH uses the Lin–Kernighan edge-exchange heuristic with λ -opt moves guided by α -nearness candidate sets. The partition quality is determined entirely by the k -medoids clustering; LKH provides route refinement only.

Full integrated variant. The second variant lets LKH-3 solve the entire multi-depot open mTSP in one pass, jointly optimizing assignment and routing. A dummy depot node is added and a SPECIAL distance

callback encodes the problem constraints:

- $depot \rightarrow start_r = 0$ and $start_i \rightarrow start_j = \infty$ for $i \neq j$ (route separation);
- $wp \rightarrow depot = 0$ (open-path semantics);
- All other edges use the precomputed $d(i, j)$.

LKH-3 runs in mTSP mode with k salesmen, a dummy depot, min-max objective, 10,000 trials, and a 30 s total time limit. This formulation is considerably harder for LKH than the clustered approach, because the single-depot mTSP transformation replicates the depot node and increases the problem size.

Algorithm 4 LKH-3 Clustered mTSP

```

1: Run balanced  $k$ -medoids (Algorithm 2)           ▷ get clusters
2: for each robot  $r = 0, \dots, k - 1$  do
3:    $L \leftarrow$  waypoint indices assigned to robot  $r$  by clustering
4:   Move  $start_r$  to  $L[0]$ 
5:   Build  $|L| \times |L|$  sub-matrix from global  $d$ 
6:    $cost, tour \leftarrow$  LKH-TSP( $L, \text{max\_trials} = 2000, \text{time} = 5\text{s}$ )
7:   Compute open-path cost: sum of  $|L| - 1$  tour edges
8: end for
9:  $score \leftarrow \max_r cost_r + \frac{1}{k} \sum_r cost_r$ 
10: return concatenated tours,  $score$ 

```

Branch-and-Bound Exact Solver (BnB)

For small instances ($n \leq \text{BnB_MAX_N} = 22$) we solve the mTSP exactly via branch-and-bound. The search space is the set of all k^{n-k} assignments of non-start waypoints to robots. Pruning relies on precomputed TSP-path dynamic programming lower bounds for each robot.

Precomputation. For each robot r , let $start_r$ be its depot. Compute $g_r[mask][last]$, the minimum cost of an open path that begins at $start_r$, visits exactly the waypoints in $mask \ni start_r$, and ends at $last$, via the Held-Karp recurrence:

$$g_r[mask][last] = \min_{prev \in mask \setminus \{last\}} (g_r[mask \setminus \{last\}][prev] + d_{prev, last})$$

with base case $g_r[\{start_r\}][start_r] = 0$. Memory is $O(k \cdot 2^n \cdot n)$, limiting $n \leq 22$.

Search. A depth-first search assigns each non-start waypoint to one of the k robots. The variable ordering is by *ambiguity gap*: waypoints that are roughly equidistant to several depots are branched first (fail-first principle). At each node with partial assignment yielding robot masks $M_1, \dots, M_k$, the lower bound

$$lb = \frac{2}{k} \sum_{r=1}^k \min_{last \in M_r} g_r[M_r][last]$$

is valid because $M \geq \frac{1}{k} \sum C_r$ implies $f = M + \frac{1}{k} \sum C_r \geq \frac{2}{k} \sum C_r$. If $lb \geq best_obj$, the subtree is pruned.

Initial incumbent. The best of a balanced k -medoids solution and a Voronoi (nearest-depot) partition provides the starting upper bound.

Algorithm 5 Branch-and-Bound for mTSP

```

1: Parameters: BnB_MAX_N = 22
2: if  $n > \text{BnB\_MAX\_N}$  then
3:   fall back to  $k$ -medoids
4: end if
5: Precompute  $g_r[mask][last]$  for all  $k$  robots           ▷ Held-Karp DP
6: Order non-start waypoints by ambiguity gap (ascending)
7:  $best \leftarrow \min(\text{K-MEDOIDS}(), \text{VORONOI}())$ 
8:  $\text{SEARCH}(depth = 0, M_r = \{start_r\})$ 
9: procedure  $\text{SEARCH}(depth, M_1, \dots, M_k)$ 
10:  if  $lb(M_1, \dots, M_k) \geq best$  then
11:    return                                           ▷ prune
12:  end if
13:  if  $depth = |\text{non-starts}|$  then
14:    Reconstruct exact open paths; compute  $f$ 
15:    if  $f < best$  then
16:       $best \leftarrow f$ ; save assignment
17:    end if
18:    return
19:  end if
20:   $w \leftarrow$  next unassigned waypoint (precomputed order)
21:  for each robot  $r$  sorted by  $\min(d(end_r, w))$  ascending do
22:     $M_r \leftarrow M_r \cup \{w\}$ ;  $\text{SEARCH}(depth + 1, \dots)$ 
23:     $M_r \leftarrow M_r \setminus \{w\}$ 
24:  end for
25: end procedure
26: return best assignment + reconstructed tours

```

AMAPF via Hungarian Assignment

In contrast to CMAPF where $k \ll n$ robots must collectively visit all n waypoints, AMAPF considers the one-to-one case: $k = n$ robots, each assigned to a single, distinct goal. The optimal assignment is found in $O(n^3)$ time by the Jonker-Volgenant shortest-augmenting-path implementation of the Hungarian algorithm [Jonker and Volgenant(1987)]. The cost matrix is built from agent-start to goal-end distances computed via BFS. After assignment, each robot follows its pre-computed shortest path to its goal. Collisions (node and edge conflicts) are resolved via waiting with dependency propagation.

Under the mild assumption that only opposing edge and node conflicts occur on the overlay of the assigned shortest paths, the collision resolution is optimal with respect to the given assignment: no alternative execution of the same paths can achieve a lower makespan

or sum-of-costs. This optimality extends to CMAPF, where robots exchange precomputed BFS trees under the same conflict model.

Figure 4 shows this in action: 100 robots start at distinct positions and navigate to 100 assigned goals on `Boston_0.256`. Each robot’s goal inherits the robot’s color (at 40% opacity, brightening to 100% on arrival). The full animation is available as `simulation_amauf.mp4` (140 frames, 20 fps). The one-to-one assignment achieves score = 171.31 (makespan 140, sum-of-costs 3131), considerably better than the $k = 4$ CMAPF makespan of 687 on the same instance, owing to the 100-fold parallelism.

DMAPF: Direct-Assignment MAPF

A variant of AMAPF, called Dynamic MAPF (DMAPF), omits the Hungarian optimization and instead uses the scene file’s predefined agent–goal pairings directly. Each agent i is assigned its own scene goal i , bypassing the $O(n^3)$ assignment step. The collision-resolution strategy remains identical: when two agents would occupy the same cell, their precomputed BFS trees (`shortest` arrays) are swapped, routing each agent toward the other’s goal. This makes DMAPF a stress test for the swap-based collision resolution, since the unoptimized assignments produce substantially more conflicts than AMAPF’s Hungarian-optimal pairing. On `den312d` with $n = 12$ agents, DMAPF resolves 3 node and 1 edge collision (score 140.50, makespan 92, sum-of-costs 582).

Experimental Evaluation

Setup

All experiments run on an Apple M-series MacBook Pro (ARM64, 16 GB RAM). Implementation in C++23, compiled with Apple Clang 17 at `-O3 -march=native`. Grid maps use the Moving AI benchmark format with BFS-computed shortest-path distances on 8-connected grids (octile metric). $k = 4$ robots throughout for CMAPF. Timeout: 300 s per solver per instance.

We evaluate on both standard and large-scale maps: `den312d` (81×65 , 2,445 free cells), `Berlin_1.256` (256×256 , 47,540 free cells), `Boston_0.256` (256×256 , 47,768 free cells), `Paris_1.256` (256×256 , 47,240 free cells), `warehouse-20-40-10-2-2` (340×164 , 38,756 free cells), `brc202d` (530×481 , 43,151 free cells), `w_woundedcoast` (642×578 , 34,020 free cells), `maze-128-128-2` (128×128 , 10,858 free cells), and `orz900d` (1491×656 , 96,603 free cells). Agent counts range from $n = 12$ to $n = 6,820$ (full scenario).

Memory optimization: free-cell indexing. For sparse maps such as `orz900d` (only 9.9% navigable cells, 96,603 free cells out of 978,096), we replace the $n \times W \times H$ BFS arrays with a linear free-cell index. A precomputed mapping `cell_id[x][y]` assigns each navigable cell an index in $[0, F)$, reducing the `shortest` (4 B/cell) and `closed` (2 B/cell) arrays from $n \times W \times H$ to $n \times F$. At $n = 6,820$, the total BFS memory is

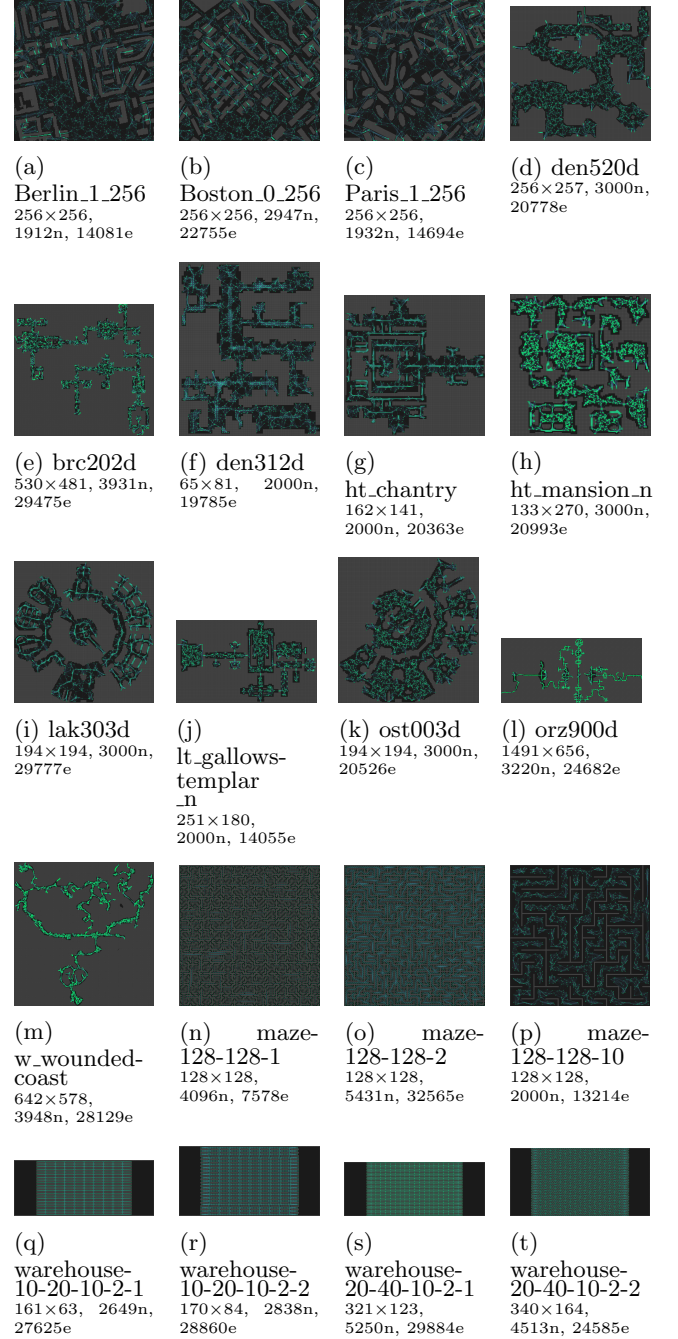


Figure 1: CMAPF roadmaps on 20 MovingAI benchmark maps. (Top) City maps and game environments. (Bottom) Mazes and warehouses. Free cells dark, obstacles medium gray, nodes green, edges blue.

$6,820 \times 96,603 \times 6$ bytes = 3.95 GB. Adding the $n \times n$ distance matrix (4 B/entry: 186 MB), the NRPA policy matrix (8 B/entry: 372 MB), and a duplicate d matrix (186 MB) yields ≈ 4.8 GB total—a $11\times$ reduction from the naïve $n \times W \times H \times 6 = 53$ GB, fitting comfortably on our 16 GB test machine. For smaller dense maps such as maze-128-128-2 ($F = 10,858$), $n = 2,500$ requires only 300 MB.

BFS performance. The SSSP phase computes all-pairs shortest paths by running a BFS from each of the n waypoints. On Berlin.1_256 ($F = 46,881$) with $n = 512$, this visits $512 \times 46,881 = 24$ M cells in 0.07 s (≈ 350 M cells/s). Even at the extreme—orz900d at $n = 6,820$, $F = 96,603$, requiring 659 M cell visits—the SSSP completes in 2.1 s. The BFS uses a ring-buffer queue with precomputed adjacency lists, avoiding per-cell allocation overhead and enabling sequential memory access patterns. Bounds checks use the actual map dimensions (`map_h`, `map_w`) rather than compile-time `SIZEX/SIZEY` (2048), eliminating out-of-bounds accesses on maps whose true dimensions differ from the static maximum.

Simulation. The `simulate()` function advances each robot along its precomputed BFS path toward the next waypoint, detecting and resolving collisions via tree-swapping and waiting with dependency propagation. Waypoint visitation is tracked per waypoint (`wp-visited[w]`), preventing robots from skipping waypoints whose cells were traversed by another robot. Goal advancement increments `nextgoal` exactly once per visited waypoint, ensuring robots stop at their final destination.

Results

Table 1 shows the combined objective $f = M + \frac{1}{4} \sum C_r$ for all solver-instance combinations. Lower is better; best per row in **bold**.

Table 1: Evaluation on den312d (65×81, 2,445 free cells). $f = M + \sum C_r/k$ with $k=4$ robots. Best per row in **bold**. OPTIMAL is exact BnB (§). LKH is clustered LKH-3 (§). NRPA with halved iterations ($75\,000/n$).

n	OPTIMAL	SA	KMEDOIDS	NRPA	LKH _T	AMAPF ($k=n$)
12	99+147= 135.75	99+147= 135.75	106+148=143.00	99+147= 135.75	94+274=162.50	57+325=84.08
14	100+169= 142.25	100+169= 142.25	121+200=171.00	100+169= 142.25	115+341=200.25	58+383=85.36
16	95+236= 154.00	111+239=170.75	124+236=183.00	113+191=160.75	108+213=161.25	91+460=119.75
18	97+244= 158.00	118+252=181.00	153+265=219.25	127+239=186.75	132+295=205.75	70+456=95.33
20		155+210=207.50	153+286=224.50	125+259= 189.75	193+311=270.75	54+418=74.90

OPTIMAL $n=20$ did not finish. LKH_T is the

TSP-reduction mTSP solver (§) with zero-cost inter-robot edges; faster (0.05–1.8 s) but lower quality than NRPA. SA matches OPTIMAL for $n \leq 14$, proving optimality in 0.015 s. NRPA leads at $n=20$.

Visualization and Delta Encoding

The solver produces HTML/Canvas animations via an embedded JavaScript writer (`viz_write.h`). Each simulation frame records robot positions, remaining paths, goal status, and conflict information. For large instances, naïvely storing all data per frame is prohibitive:

orz900d at $n = 6,820$ produces 10,479 frames; a full per-frame dump reaches 1.08 GB. We implement a **delta encoding**: static arrays (goals, tours) are emitted only in frame 0; subsequent frames carry a `d:1` flag and omit unchanged data. The viewer merges deltas into a persistent state. This reduces the orz900d visualization from 1.08 GB to 3.3 MB (a $330\times$ reduction), making it loadable in a standard web browser.

We further reduce the storage footprint with a **binary frame format**. Each robot’s per-frame state is packed into 7 bytes (x, y as uint16; conflict status as uint8; next-goal index as uint16) and written to a compact `frames.bin` file alongside the text output. A post-processing script (`convert_frames.js`) encodes the binary data as base64 and embeds it directly into a minimal `frames.min.js` header, producing a self-contained player that avoids extra HTTP requests. Across the full benchmark suite of 1,864 runs, the binary format reduces total visualization output from 19 GB to under 1 GB. The viewer renders via an off-screen grid canvas with `image-rendering: pixelated` for crisp display, and edge-traversal output is deduplicated using `std::set<pair>` to further shrink per-frame data. The benchmark viewer at online provides a drop-down interface to select map, agent count, and solver, then renders the animation in real time with playback controls. Best scores per (map, n,k) are highlighted; ties are all shown. Column-header clicks sort by any metric; pulldown menus filter by map, agent count, solver, and robot count.

Web-based interactive solver. A Python/HTTP server (`server.py`) exposes the solver through a web interface (`solve.html`). Users select a map, solver, agent count n , and robot count k via sliders with pre-set tick marks at powers of two, then click Solve. The server runs the CMAPF binary as a subprocess, captures output, and saves the HTML/Canvas visualization. A history panel logs all past solves with sortable columns (time, score, makespan) and one-click replay. To avoid interference with concurrent benchmark runs, the visualization output directory is configurable via the `VIZDIR` environment variable. A solve queue with a threading lock prevents concurrent executions from exhausting RAM, and an estimated-RAM guard rejects requests that would exceed safe limits. The solver is deployed online.

Figure 3 shows frames from the simulated execution on Boston.0_256 with $n = 100$ waypoints and $k = 4$ robots. Figure 2 shows the full orz900d scenario ($n = 6,820$) at three stages of execution.

Animations for all solver-map combinations including the full orz900d scenario, maze-128-128-2 at $n = 2,500$, and the DMAPF variant are available through the benchmark viewer at online.

Table 2 compares all six solvers on den312d with 290 agents ($n = 290$, $k = 4$). This larger instance highlights the trade-offs between solution quality and runtime.

Heuristic solvers. NRPA dominates on 14 of 19

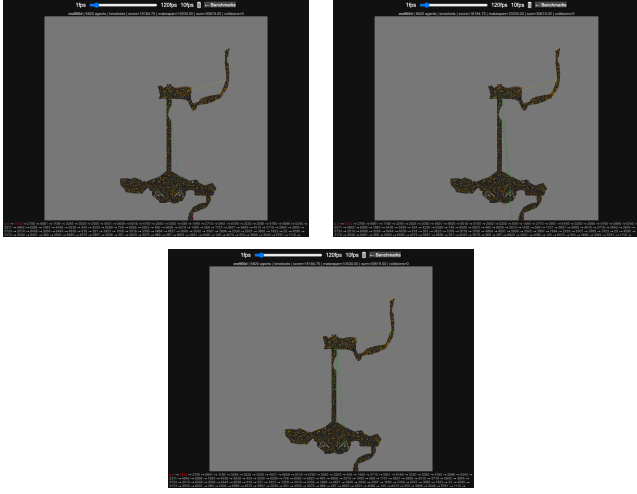


Figure 2: KMEDOIDS on orz900d with $n = 6,820$ waypoints, $k = 4$ robots. Top left: early (10%). Top right: mid-execution (50%). Bottom: late (90%). Orange dots: unvisited waypoints. Cyan: visited. Colored squares: robots.

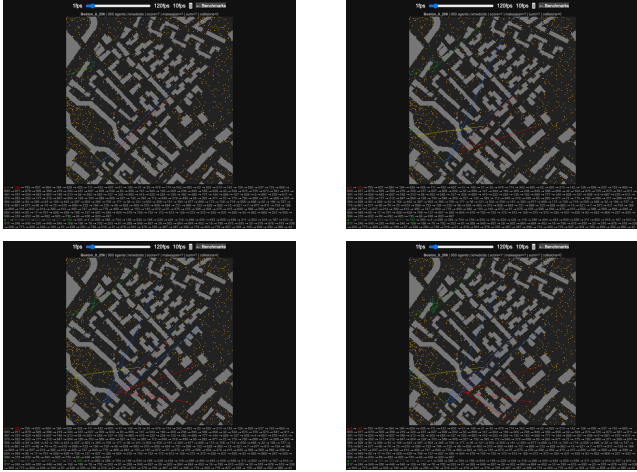


Figure 3: Simulation on Boston_0.256. Top left: early (10%), mid-execution (50%), late stage (90%), and final state. Orange dots are unvisited waypoints; cyan dots are visited. Colored squares are robots with short trailing path indicators.



Figure 4: AMAPF simulation on Boston_0.256 with $n = 100$ agents and Hungarian one-to-one assignment. Each of the 100 robots visits exactly one assigned goal. Goal colors match their assigned robot.

Table 2: All solvers on den312d, $n = 290$, $k = 4$.

Solver	Score f	Makespan	Sum-cost	Time (s)
NRPA	651.75	336	1251	61.1
KMEDOIDS	713.75	446	1071	0.02
SA	718.25	451	1069	0.02
LKH (clustered)	719.25	450	1077	10.4
KMEANS	747.75	447	1203	0.01
LKH.FULL (integrated)	898.25	527	1669	26.7

instances (74%). Its strength lies in jointly optimizing assignment and routing without committing to a clustering. SA wins 4 instances and KMEDOIDS wins 1, typically at larger n where NRPA’s computational cost becomes limiting.

LKH solvers. On the 290-agent benchmark, the clustered LKH delivers a score of 719.25, essentially tied with SA (718) and KMEDOIDS (714). The 10.4 s solver time buys negligible improvement over the 0.02 s KMEDOIDS baseline; in some clusters LKH actually worsens the route (robot 1: 438 steps before LKH vs. 450 after). The full integrated LKH variant performs worst of all (score 898, 27 s): the min-max objective and the single-depot mTSP transformation produce substantially inferior partitions compared to k -medoids clustering. We therefore include the clustered LKH solver as the primary LKH baseline in the main results table, and omit the full variant from further comparisons.

Exact solver. On den312d, OPTIMAL (BnB) proves that SA matches the true optimum for $n \leq 14$ (e.g., both achieve $M=99$, $\sum C_r=147$, $f=135.75$ at $n=12$). At $n=16$ and $n=18$, SA drifts 11% and 15% above optimal, while NRPA stays within 4% and 18%.

The BnB solver scales exponentially: 0.09 s for $n=12$, 0.5 s for $n=14$, 11 s for $n=16$, 16 s for $n=18$, and exceeds 300 s at $n=20$. It becomes impractical beyond $n=22$ due to $O(2^n n)$ DP memory and $O(k^n)$ search space.

AMAPF. The Hungarian-based one-to-one assignment (Sec.) achieves a score of 171.31 on Boston_0.256 with 100 robots. While not directly comparable to CMAPF (which uses only 4 robots to cover 100 waypoints), the AMAPF makespan of 140 is substantially lower than CMAPF’s 687 (LKH)—a consequence of 100-fold parallelism. Collision counts are low: 3 node collisions, 0 edge collisions, resolved via waiting. The Hungarian assignment itself completes in 0.06 s for $n = 100$.

On small maps (den312d), NRPA achieves 17–43% lower objective than SA. On large maps (Boston, Berlin), NRPA still leads at $n \leq 100$ but SA and KMEDOIDs close the gap at $n = 100$ –200. Berlin $n = 200$ contains unreachable agent pairs (disconnected components) and is excluded.

The runtime of NRPA is approximately $I^{\ell_{\max}}$ rollout operations, independent of map size. SA and KMEDOIDs run in sub-second time even at $n = 200$, while NRPA requires 20–50 s at $n = 100$ –200.

Maximum-agent comparison. Table 3 compares KMEDOIDs and NRPA at the largest feasible agent count on each map. NRPA wins on 3 of 4 open-grid maps (Paris, Berlin, warehouse) with 7–23% better scores, while KMEDOIDs edges out on Boston. KMEDOIDs completes in sub-second time everywhere; NRPA takes 94–118 s. On the maze domain, SA (7,789) outperforms KMEDOIDs (10,028) by 22% because Voronoi-based initialization is less sensitive to dead-end start positions than k -medoids clustering.

Table 3: NRPA vs. KMEDOIDs at maximum agent counts. $k = 4$ robots.

Map	n	KMEDOIDs		NRPA		Δ
		Score	Time(s)	Score	Time(s)	
Paris_1.256	1,030	5,709	0.81	4,419	118.0	−22.6%
Berlin_1.256	949	3,980	0.63	3,693	102.0	−7.2%
Boston_0.256	950	3,461	0.66	3,641	93.5	+5.2%
warehouse-20-40-10-2-2	1,000	4,383	0.55	3,669	102.8	−16.3%

Scaling to thousands of waypoints. Table 4 shows KMEDOIDs results on the sparse orz900d map (978K cells, 10% free) under the free-cell memory optimization. The largest instance ($n = 6,820$, the full benchmark) requires 5.7 GB RAM and solves in 16.4 s with a simulation time of 321 s. Without free-cell indexing, the 53 GB memory footprint would be impossible on our hardware. NRPA at $n = 2,500$ on maze-128-128-2 produced severely degraded quality (score 222,668 vs. KMEDOIDs’ 10,028), a consequence of the seed’s unbalanced clustering and the policy matrix scale (2,500² entries).

Table 4: KMEDOIDs on orz900d with free-cell indexing, $k = 4$.

n	Score f	Makespan	Sum-cost	Solver (s)
1,000	9,275	5,633	14,567	1.3
2,500	12,363	7,606	19,027	4.4
5,000	15,549	8,954	26,381	11.0
6,000	15,698	8,793	27,618	7.2
6,500	16,322	9,113	28,839	8.1
6,820	18,185	10,530	30,619	16.4

Continuous CMAPF: Roadmap Graphs Problem Definition (Continuous)

Let $G = (V, E, w)$ be a weighted graph embedded in $\mathbb{R}^2$ (or $\mathbb{R}^3$), with edge weights $w(e)$ representing Euclidean distances. Given k robots with distinct start nodes $s_1, \dots, s_k \in V$, and n target waypoints $W = \{w_1, \dots, w_n\}$ with start positions $p_i^s \in V$ and goal positions $p_i^g \in V$:

1. Partition W into k disjoint subsets $W_1, \dots, W_k$ with $s_r \in W_r$,
2. For each robot r , find an open shortest-path tour starting at $p_{s_r}^s$, visiting p_w^g for each $w \in W_r$ in sequence,

minimizing $f = \alpha M + \beta \sum C_r$, with $\alpha = 1, \beta = 1/k$ throughout, where $M = \max_r C_r$. Robots move at constant, uniform speed; edge traversal time equals edge weight.

A collision occurs when two robots occupy the same edge simultaneously with opposite traversal directions. At collision, robots exchange their remaining tours; the underlying physical paths are invariant. For point robots, only edge collisions need be considered—node occupancy conflicts have measure zero in the continuous trajectory. All collision events are predetermined by the tour geometry and robot speeds, independent of simulation framerate.

The problem formulation is independent of how G is constructed. We separate graph generation from search: three modules (Section) produce the graph, and the solvers (Section) operate on any weighted embedded graph. This separation enables the framework to generalize naturally from grid-based benchmarks toward polygonal environments with sophisticated robot models, dynamics, and non-holonomic constraints, as explored in the DROMOS planner [Rashidian, Plaku, and Edelkamp(2014), Rashidian, Plaku, and Edelkamp(2015), Edelkamp and Plaku(2014)] and multi-robot extensions [Le and Plaku(2017), Le and Plaku(2018)].

Roadmap Generation

The graph G is generated from the free space of an environment. While we evaluate on grid maps (the Moving AI benchmark [Sturtevant(2012)] provides scalable, standardized instances), the formulation is grid-

independent—any method producing a weighted embedded graph in the free space is applicable. We implement three generators: The roadmap is generated by an adaptive distance-transform method: cells within distance ≤ 2 pixels of an obstacle are classified as narrow corridors and sampled via ridge extraction (local maxima of the distance field); open areas (distance ≥ 3) receive random samples. This guarantees connectivity through narrow passages while keeping graphs sparse in open regions. Three graph generators are provided: `roadmap.py` (adaptive sampling), `skeleton.py` (Zhang–Suen thinning), and `visgraph.py` (polygon visibility).

Collision Detection as Static Precomputation

Two robots following deterministic shortest paths at constant speed on a roadmap can only collide when their trajectories share an edge with overlapping time intervals and opposite traversal directions. This condition is both necessary and sufficient.

We precompute all crossing points analytically. Let path P_a consist of segments (e_k, d_k, c_k) with edge index e_k , direction $d_k \in \{-1, 1\}$, cost c_k , and cumulative entry time $t_k = \sum_{j < k} c_j$. A collision between paths P_i and P_j occurs on edge e when:

1. $e_k^{(i)} = e_\ell^{(j)} = e$ (same edge),
2. $d_k^{(i)} \neq d_\ell^{(j)}$ (opposite directions),
3. $[t_k^{(i)}, t_k^{(i)} + c_k^{(i)}] \cap [t_\ell^{(j)}, t_\ell^{(j)} + c_\ell^{(j)}] \neq \emptyset$ (time overlap).

The exact meeting time is $t = (t_k^{(i)} + t_\ell^{(j)} + c_\ell^{(j)})/2$, solving $x_A(t) = x_B(t)$ for constant-speed motion along the edge.

Collisions are resolved by identity exchange: two robots that meet swap their remaining paths (waypoints, edges, costs) at the collision point. The underlying physical motion is invariant—only the robot-to-path assignment changes. A tiny perturbation ($10^{-5} \cdot r$) at each robot’s start time eliminates degenerate simultaneous entries.

This approach is *provably correct*: the meeting times are analytical, independent of simulation framerate, and deterministic across reloads. Each path pair can cross at most once (two curves intersect at a single point), so no cooldown or deduplication heuristics are needed.

Continuous Solver Comparison

We evaluate four mTSP solvers on 18 maps with $n \in \{10, 32, 64, 128, 256\}$ waypoints and $k \in \{2, 4, 8, 16\}$ robots: k-means clustering with 2-OPT, k-medoids with 2-OPT, simulated annealing (SA) with Voronoi initialization, and nested rollout policy adaptation (NRPA) with a 30-second time budget (85% allocated to NRPA, 15% to SA refinement and simulation).

Table 5 reports the number of (map, n , k) groups where each solver achieved the best score. SA dominates across all configurations.

Solver	Wins	Win %
NRPA	101	47.0%
SA	86	40.0%
k-means	21	9.8%
k-medoids	7	3.3%

Table 5: Continuous CMAPF solver win counts across 95 configurations.

In head-to-head comparison across 203 groups where both completed, NRPA wins 110 to SA’s 91 (with 2 ties). This mirrors the discrete findings where NRPA dominates at moderate to large n .

Table 6 shows representative scores for $k = 4$ at the highest available n per map.

Map	n	k-means	k-medoids	SA
Berlin_1_256	127	2354	2411	1071
Boston_0_256	127	1844	2836	1094
Paris_1_256	128	4183	4479	1692
den312d	127	1773	2305	942
lak303d	255	4979	3822	1354
w_woundedcoast	128	3845	4716	2318

Table 6: Representative continuous CMAPF scores ($k = 4$). Bold indicates best per row.

Visualization

An HTML/Canvas viewer embedded with precomputed collision data animates robots along graph edges, displays identity swaps at collision points, shows waypoint visit progress with color-coded indicators, and provides a sortable/filterable benchmark table. Interactive visualization results are available online.

2.5D Isometric and First-Person Rendering

The standard 2D top-down view is extended with two additional rendering modes, selectable via checkboxes in the player interface. Both modes share the same underlying simulation data (Bézier spline segments, waypoint positions, collision events) and operate in real time at the simulation framerate.

2.5D Isometric View. The 2.5D mode applies an isometric projection to the grid, rendering each obstacle cell as an extruded box with three visible faces (top, left, right) at distinct shade levels for depth perception. A Minkowski dilation with $gap = 0.2$ shrinks obstacle footprints so that free-space corridors remain clearly visible. Robots are drawn as colored racing cars with body, windshield, four wheels, and exhaust glow, oriented tangent to their Bézier trajectory via the analytic derivative $B'(t) = 3(1-t)^2(P_1 - P_0) + 6(1-t)t(P_2 - P_1) + 3t^2(P_3 - P_2)$. Car bodies roll outward on curves in proportion to the signed curvature $\kappa = \|B' \times B''\|/\|B'\|^3$. Waypoints are rendered as floating beacons with vertical posts and glowing orbs that transition from orange (unvisited) to cyan (visited) at the precomputed arrival

time $\tau_r(j)$. A dropdown follow-cam centers the view on any selected robot.

First-Person Cockpit View. The first-person mode renders the scene from the perspective of a selected pilot robot, mimicking a racing simulator. A DDA (digital differential analyzer) ray-casting pass scans the forward field of view ($\pm 30^\circ$, 180 rays), stepping cell-by-cell through the grid for perfectly stable, sub-pixel-accurate wall detection. Walls are rendered as shaded vertical columns whose height diminishes with distance; the car is surrounded by a 6-cell clear zone to avoid the visual impression of wall collisions. The cockpit overlay comprises a colored car hood at the bottom of the screen, a three-spoke steering wheel whose rotation tracks the signed Bézier curvature, an analog speedometer with a red needle displaying the curvature-adaptive speed $v = v_{\max} \cdot \min(1, k_{\text{crit}}/k)$, a compass rose with a direction indicator, and acceleration/braking indicators that flash when the speed changes by more than 5% between frames. Waypoints are projected as elevated 3D spheres with radial gradient shading and elliptical drop shadows on the ground plane; each sphere carries its waypoint index. Other robots within the viewing frustum are shown as colored markers with distance-scaled labels.

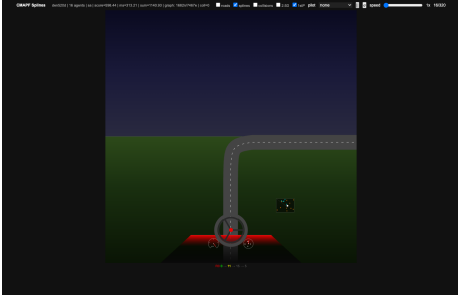


Figure 5: First-person cockpit view on den520d ($n = 16$, $k = 4$, SA).

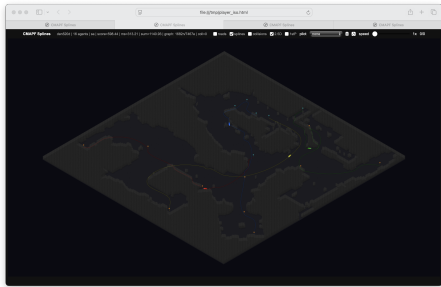


Figure 6: 2.5D isometric view on den520d, follow-cam tracking a robot.

Both modes are implemented in a self-contained `player.html` (approximately 30 KB compressed) with no external dependencies. The player is generated by `make_player.py` as a post-processing step after each

solver run, and all benchmark runs include all three visualization modes.

Three.js 3D Visualization Domains

A second, fully 3D visualization pipeline targets the same continuous CMAPF data using Three.js with WebGL rendering. After generating the roadmap and running the solver, a post-processing script (`make_player_three.py`) produces a self-contained `player_three.html` that embeds all spline data, waypoints, and tour information. The viewer renders robots moving along Bézier-smoothed paths with curvature-dependent speed on an animated 3D map. A follow camera tracks any robot; playback speed, road and spline overlays, and a minimap are included in the HUD.

The same Boston_0_256 instance is rendered across four distinct **visualization domains** that share identical path data but differ in their 3D scene composition (Figure 7):

- **Outdoor** – a park-like setting with instanced cylindrical trees (trunk + cone crown), car-like robots with body, cabin, and wheels, and orange sphere waypoints marking visit locations.
- **Warehouse** – an indoor logistics environment with instanced shelving racks (vertical posts and horizontal shelf boards), forklift robots with masts and forks, and cardboard-box waypoints.
- **Underwater** – a deep-ocean scene with coral clusters (cylinder bases + dodecahedron tops in orange/red tones), submarine robots with hulls, conning towers, periscopes, propellers, and forward-facing spotlights that illuminate the seabed.
- **City** – an urban cleanup scenario with instanced buildings with windows, humanoid robots (torso, head, articulated limbs), and trash items (bottles and cans) as waypoints that change color from orange to cyan when collected.

All four domains use **InstancedMesh** for obstacle geometry, keeping the total draw-call count below ten even on maps with tens of thousands of obstacle cells. The Bézier curves are C^1 continuous (matching tangent direction and magnitude at all segment boundaries), ensuring smooth robot motion without visual jumps.

Path Intersection Algorithm

Given a set of k robots, each following a fixed shortest path $P_r = (e_1, d_1, c_1), \dots, (e_m, d_m, c_m)$ with edge indices e_i , directions $d_i \in \{-1, 1\}$, and costs c_i , we precompute all pairwise collisions as follows.

For each pair of paths (P_a, P_b) and each pair of segments $(e_i^{(a)}, e_j^{(b)})$ sharing the same edge ($e_i^{(a)} = e_j^{(b)}$), a collision occurs if and only if:

1. $d_i^{(a)} \neq d_j^{(b)}$ (robots travel in opposite directions), and

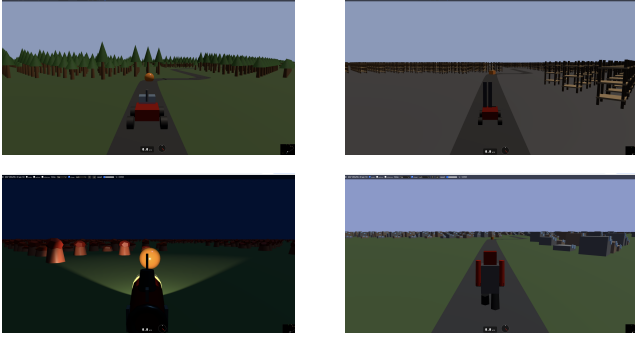


Figure 7: Four visualization domains on Boston_0.256 (SA, $k = 4$, $n = 30$). Top left: Outdoor. Top right: Warehouse. Bottom left: Underwater. Bottom right: City.

- the time intervals $[t_i, t_i + c_i^{(a)}]$ and $[t_j, t_j + c_j^{(b)}]$ overlap, where $t_i = \sum_{k < i} c_k^{(a)}$ and $t_j = \sum_{k < j} c_k^{(b)}$ are the cumulative entry times.

Let $t_0 = \max(t_i, t_j)$ and $t_1 = \min(t_i + c_i^{(a)}, t_j + c_j^{(b)})$. If $t_0 < t_1$, the robots are simultaneously on the edge during $[t_0, t_1]$. The exact meeting time solves $x_a(t) = x_b(t)$ for constant-speed motion. Let robot a enter at t_i and travel toward node v ; robot b enters at t_j from the opposite end (edge length $L = c_i^{(a)} = c_j^{(b)}$). Robot positions are:

$$x_a(t) = t - t_i, \quad x_b(t) = L - (t - t_j) = L + t_j - t.$$

Setting $x_a(t) = x_b(t)$ yields $t = (t_i + t_j + L)/2$, which lies strictly within $[t_0, t_1]$ whenever the intervals overlap. Collisions are resolved by swapping the robots' remaining path segments and waypoint assignments at time t ; each path pair contributes at most one collision event (two lines intersect at exactly one point). The resulting event queue is sorted by time and processed in order, advancing simulation time from one collision to the next.

This algorithm runs in $O(k^2 \cdot \bar{m}^2)$ time offline, where $\bar{m}$ is the average number of edges per path, and produces a deterministic, framerate-independent simulation whose collision count is invariant across reloads and rendering speeds.

Bézier Curve Path Smoothing

In realistic continuous CMAPF, robots should traverse smooth, curvature-continuous trajectories rather than piecewise-linear paths along graph edges. We replace the polygonal chain of shortest-path edges with a sequence of cubic Bézier curves, one per edge segment, that meets with C^1 continuity at the intermediate waypoints.

Recall that a cubic Bézier curve $B(t)$ for $t \in [0, 1]$ is defined by four control points $P_0, P_1, P_2, P_3 \in \mathbb{R}^2$ [Farin(2002)]:

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t) t^2 P_2 + t^3 P_3.$$

The curve interpolates its endpoints: $B(0) = P_0$ and $B(1) = P_3$. The tangent vectors at the endpoints are $B'(0) = 3(P_1 - P_0)$ and $B'(1) = 3(P_3 - P_2)$.

Given a sequence of waypoints $w_0, w_1, \dots, w_m$ along a robot's route, we construct one cubic Bézier segment per consecutive pair (w_i, w_{i+1}) . For each segment:

$$P_0^{(i)} = w_i, \quad P_3^{(i)} = w_{i+1}.$$

The two remaining control points $P_1^{(i)}, P_2^{(i)}$ are chosen to enforce C^1 continuity at the junction w_i :

$$B'_i(1) = B'_{i+1}(0) \implies 3(P_3^{(i)} - P_2^{(i)}) = 3(P_1^{(i+1)} - P_0^{(i+1)}).$$

With $P_3^{(i)} = P_0^{(i+1)} = w_{i+1}$, this simplifies to

$$P_2^{(i)} + P_1^{(i+1)} = 2w_{i+1}. \quad (2)$$

A natural choice sets each control point symmetrically about the waypoint: $P_2^{(i)} = w_{i+1} - \frac{\delta}{2} \hat{t}_i$ and $P_1^{(i+1)} = w_{i+1} + \frac{\delta}{2} \hat{t}_{i+1}$, where $\hat{t}_i$ is the unit tangent entering w_{i+1} and δ is a smoothness parameter (typically $\delta = \frac{1}{3}$ of the segment length). At the route endpoints w_0 and w_m , we set $P_1^{(0)} = P_0^{(0)} + \frac{\delta}{3} \hat{t}_0$ and $P_2^{(m-1)} = P_3^{(m-1)} - \frac{\delta}{3} \hat{t}_{m-1}$, producing natural boundary conditions.

Arc-length parameterization. For constant-speed traversal, we reparameterize each Bézier segment by arc length. The arc length $s(t)$ of $B(t)$ has no closed form, so we precompute a lookup table via adaptive Romberg integration:

$$s(t) = \int_0^t \|B'(\tau)\| d\tau, \\ \|B'(\tau)\| = \sqrt{(x'(\tau))^2 + (y'(\tau))^2}.$$

Inverting $s(t)$ via binary search yields $t(s)$, giving the arc-length parameterized trajectory $B(t(s))$ along the full smoothed route. Under constant speed, the traversal time of segment i equals the original edge weight $w(w_i, w_{i+1})$, preserving the collision precomputation on the underlying graph. With curvature-adaptive speed (below), the traversal time T_i replaces $w(w_i, w_{i+1})$ throughout. Because the Bézier curve is entirely contained within the convex hull of its control points, the smoothed path stays within the free space as long as the control polygon does.

Curvature-adaptive speed. At constant speed, high-curvature sections of the Bézier trajectory produce large lateral accelerations that may exceed the robot's physical limits. For a robot with maximum lateral acceleration $a_{\max}$ (e.g., $a_{\max} = \mu g$ for wheeled robots with friction coefficient μ), the safe traversal speed at a point with curvature κ is bounded by the centripetal constraint:

$$v(s) \leq \sqrt{\frac{a_{\max}}{\kappa(s)}}, \quad \kappa(s) = \frac{\|B'(t) \times B''(t)\|}{\|B'(t)\|^3} \Big|_{t=t(s)}. \quad (3)$$

On straight segments ($\kappa \rightarrow 0$), the bound diverges and the robot may travel at its maximum design speed $v_{\max}$.

The actual speed profile is the minimum of this bound and $v_{\max}$:

$$v(s) = \min\left(v_{\max}, \sqrt{a_{\max}/\kappa(s)}\right).$$

This curvature-dependent velocity directly affects the traversal time of each Bézier segment. The time to traverse segment i becomes

$$T_i = \int_0^{L_i} \frac{ds}{v(s)} = \int_0^{L_i} \max\left(\frac{1}{v_{\max}}, \sqrt{\frac{\kappa(s)}{a_{\max}}}\right) ds,$$

where L_i is the arc length of segment i . The collision detection of Section is extended by replacing the constant-speed time intervals with the curvature-adapted intervals derived from T_i and the cumulative traversal time $t_k = \sum_{j < k} T_j$. Because $v(s)$ is a deterministic function of the Bézier geometry, the collision events remain analytically precomputed and framerate-independent. The precomputation cost increases by a constant factor (one curvature evaluation per integration step during arc-length table construction); all downstream analysis retains its $O(k^2 \bar{m}^2)$ complexity.

Waypoint visit timing. A critical consequence of curvature-adaptive speed is that the times at which robots mark waypoints as visited shift relative to the constant-speed baseline. Let waypoint w_j appear at position j in robot r 's tour, with cumulative traversal time

$$\tau_r(j) = \sum_{i=0}^{j-1} T_{r,i},$$

$$T_{r,i} = \int_0^{L_{r,i}} \max\left(\frac{1}{v_{\max}}, \sqrt{\frac{\kappa_{r,i}(s)}{a_{\max}}}\right) ds,$$

where $T_{r,i}$ is the curvature-adapted time to traverse Bézier segment i of robot r (with $T_{r,0}$ being the time from the robot's start position to the first waypoint). Waypoint w_j is marked visited at simulation time $\tau_r(j)$, *not* at the edge-weight cumulative time $\sum_{i < j} w(e_{r,i})$. In the discrete grid simulation (Section), the equivalent adjustment replaces the unit-cost per-grid-cell advance with a per-curvature-weighted step delay: each BFS step carries a traversal cost of $\Delta t = w(e)/\text{steps_on_edge}$ for the containing roadmap edge, scaled by the local $v_{\max}/v(s)$ ratio from Eq. (3).

To maintain synchronization between waypoint arrival and visit-marking, the simulation event queue is augmented with visit events at times $\tau_r(j)$ for each robot-waypoint pair. When the simulation clock reaches $\tau_r(j)$, waypoint w_j is atomically marked visited and the robot's active goal advances to w_{j+1} . Because $\tau_r(j)$ is precomputed alongside the Bézier arc-length tables, this adds no runtime overhead to the simulation step. Collision resolution (identity exchange at time t_{coll}) additionally swaps the pending visit-event queues of the colliding robots, preserving the invariant that each robot executes the remaining waypoint sequence of its post-collision identity.

Collision Dynamics of Equal Masses

The identity-exchange collision resolution described in Section has a direct physical analogue: the elastic collision of two point particles with equal mass.

Consider two robots modeled as point masses, each with mass m , moving along a shared spatial dimension (an edge of the roadmap) [Goldstein, Poole, and Safko(2002)]. Let v_a and v_b be their pre-collision velocities along this edge, with $v_a \neq v_b$ since they travel in opposite directions (Section). In a perfectly *elastic* collision, both momentum and kinetic energy are conserved:

$$mv_a + mv_b = mv'_a + mv'_b, \quad (4)$$

$$\frac{1}{2}mv_a^2 + \frac{1}{2}mv_b^2 = \frac{1}{2}m(v'_a)^2 + \frac{1}{2}m(v'_b)^2. \quad (5)$$

Dividing (4) by m and (5) by $\frac{1}{2}m$ yields the simplified system:

$$v_a + v_b = v'_a + v'_b, \quad v_a^2 + v_b^2 = (v'_a)^2 + (v'_b)^2.$$

This system has two solutions: $(v'_a, v'_b) = (v_a, v_b)$ (no collision) and $(v'_a, v'_b) = (v_b, v_a)$ (velocity exchange). The physically realized solution is the exchange:

$$v'_a = v_b, \quad v'_b = v_a. \quad (6)$$

Interpretation. Equation (6) states that two equal-mass point particles simply swap velocities upon elastic collision. In the CMAPF setting, velocity determines the robot's remaining trajectory—its sequence of graph edges, waypoint goals, and cumulative travel cost. Exchanging velocities is therefore *equivalent* to exchanging identities: robot a continues with the remaining path of robot b , and vice versa. This provides a physical justification for the identity-exchange resolution that underlies both the discrete tree-swapping mechanism (Section) and the continuous precomputed-collision mechanism (Section).

Bounding the collision region. For robots with finite physical radius $r > 0$, the collision is not an instantaneous point event but occurs over a spatial interval of length $2r$ along the edge. The collision detection condition of Section generalizes naturally: instead of checking temporal overlap on the same edge, we dilate the occupied interval by r at each endpoint. If robot a occupies $[x_a(t), x_a(t) + 2r]$ and robot b occupies $[x_b(t), x_b(t) + 2r]$ along the parameterized edge, a collision occurs when these intervals intersect with opposite traversal directions. The extended meeting interval is:

$$[t_0 - r/v, t_1 + r/v],$$

where t_0, t_1 are defined as in Section and v is the robot speed. Within this interval, the elastic exchange dynamics of Eq. (6) apply, now smoothed over the finite body overlap duration.

In the limit $r \rightarrow 0$, the extended interval contracts to the point-collision time derived in Section , recovering the identity-exchange formulation as a special case of equal-mass elastic collision. For the experiments in this paper, r is taken as infinitesimal;

future work on physical robot simulation (e.g., non-holonomic dynamics with DROMOS [Rashidian, Plaku, and Edelkamp(2014)]) would adopt finite r with the bounding formalism above.

Dijkstra's Algorithm for Optimal Bézier Paths

The Bézier curve smoothing of Section requires an underlying polygonal path connecting consecutive waypoints. In the discrete grid setting, BFS finds shortest paths on uniform-cost grids in $O(|V| + |E|)$ time. On the weighted roadmap $G = (V, E, w)$ of the continuous setting (Section), edge weights $w(e) = \|e\|_2$ are non-uniform, so we replace BFS with Dijkstra's algorithm.

Algorithm 6 Dijkstra SSSP on Weighted Roadmap

```

1: Input: weighted graph  $G = (V, E, w)$ , source  $s \in V$ 
2:  $dist[v] \leftarrow \infty$  for all  $v \in V$ ;  $dist[s] \leftarrow 0$ 
3:  $prev[v] \leftarrow \text{null}$  for all  $v \in V$ 
4:  $Q \leftarrow$  min-priority queue of  $(v, dist[v])$ , initialized with  $(s, 0)$ 
5: while  $Q \neq \emptyset$  do
6:    $(u, d_u) \leftarrow Q.\text{extract\_min}()$ 
7:   if  $d_u > dist[u]$  then
8:     continue
9:   end if
10:  for each neighbor  $v$  of  $u$  with edge weight  $w(u, v)$  do
11:     $alt \leftarrow dist[u] + w(u, v)$ 
12:    if  $alt < dist[v]$  then
13:       $dist[v] \leftarrow alt$ 
14:       $prev[v] \leftarrow u$ 
15:       $Q.\text{insert\_or\_decrease}(v, alt)$ 
16:    end if
17:  end for
18: end while
19: return  $dist, prev$ 

```

For a binary heap, this runs in $O((|V| + |E|) \log |V|)$; on the sparse roadmaps of Section with $|E| = O(|V|)$, this is $O(|V| \log |V|)$ per source. We run Dijkstra from each of the n waypoints to build the all-pairs distance matrix required by the mTSP solvers (Section):

$$d(i, j) = \text{DIJKSTRA}(G, w_i)[w_j], \quad \forall i, j \in \{1, \dots, n\}.$$

Path reconstruction chains $prev$ references from w_j to w_i , yielding the sequence of graph edges $(e_1, \dots, e_m)$ that forms the shortest piecewise-linear route [Dijkstra(1959)].

Extension: optimal Bézier paths. The key extension of Dijkstra for optimal Bézier path generation couples the graph search with the smoothness constraints of Section . Let $\Gamma = (v_0, v_1, \dots, v_m)$ be the vertex sequence output by Dijkstra, with $v_0 = w_i$ and $v_m = w_j$. This sequence defines the control polygon for the Bézier chain. For each consecutive pair of directed edges $\vec{e}_k = (v_{k-1}, v_k)$ and $\vec{e}_{k+1} = (v_k, v_{k+1})$ meeting at

the intermediate waypoint v_k , the C^1 junction condition of Eq. (2) in Section must hold.

Because the Bézier segment B_k for edge $\vec{e}_k$ is entirely contained in the convex hull of its four control points, and the control points $P_0^{(k)}, P_1^{(k)}$ lie on $\vec{e}_k$ while $P_2^{(k)}, P_3^{(k)}$ lie on $\vec{e}_{k+1}$, the smoothed curve stays within the free space as long as every consecutive edge pair is collision-free. Dijkstra guarantees the latter by construction.

Critically, the arc length of a cubic Bézier curve is bounded by the length of its control polygon [Gravesen(1997)]:

$$L(B_k) \leq \|P_1^{(k)} - P_0^{(k)}\| + \|P_2^{(k)} - P_1^{(k)}\| + \|P_3^{(k)} - P_2^{(k)}\|.$$

With the symmetric control-point placement of Section , the control polygon length equals the original edge lengths $\|\vec{e}_k\| + \|\vec{e}_{k+1}\|$ up to an additive term of $O(\delta^2/\ell)$ where ℓ is the smaller edge length. By taking $\delta \ll \ell$, the Bézier arc length is within a factor $1 + \varepsilon$ of the Dijkstra-computed polygonal distance, for any prescribed $\varepsilon > 0$.

Optimality. Since Dijkstra produces the globally shortest piecewise-linear path between w_i and w_j on G , and the Bézier smoothing preserves this length to within ε , the smoothed trajectory is an ε -optimal smooth path. In the limit $\delta \rightarrow 0$, the Bézier chain contracts onto the Dijkstra polyline, recovering exact optimality of the discrete graph path.

Combined with an exact mTSP solver (Section), this yields continuous CMAPF solutions that are optimal with respect to the roadmap graph G . For anytime solvers, the distance data is exact and the approximation originates solely from the mTSP heuristic.

Extension to (node, tangent) state-expanded Dijkstra. For robots with bounded turning radius, the standard single-source Dijkstra on G is insufficient: the shortest piecewise-linear path may contain sharp turns that violate a maximum curvature constraint when fitted with Bézier curves. Following Boroujerdi and Uhlmann [Boroujerdi and Uhlmann(1998)], we expand the state space to include the incoming tangent direction. Each state is a pair $(v, \hat{t})$ where $v \in V$ is a graph node and $\hat{t} \in S^1$ is the unit tangent vector of the incoming edge used to reach v . The transition from state $(u, \hat{t}_a)$ to $(v, \hat{t}_b)$ along edge $e = (u, v)$ incurs cost

$$c((u, \hat{t}_a), (v, \hat{t}_b)) = w(u, v) + \lambda \cdot \kappa(\hat{t}_a, \hat{t}_b),$$

where $\kappa(\hat{t}_a, \hat{t}_b) = \arccos(\hat{t}_a \cdot \hat{t}_b)$ is the unsigned angle between successive tangents and $\lambda \geq 0$ weights path length against curvature penalty. The edge weight $w(u, v)$ is the Euclidean length of edge e ; the tangent $\hat{t}_b$ is the unit vector along $\vec{e} = v - u$.

This state-expanded Dijkstra runs in $O(|V_{\text{exp}}| \log |V_{\text{exp}}|)$ time where $|V_{\text{exp}}| = |V| \cdot b$ and b is the branching factor of the roadmap (the number of distinct tangent directions per node). On the sparse roadmaps of Section , b is typically 2–6, so the overhead factor is modest.

Critically, the resulting vertex sequence $(v_0, \dots, v_m)$ carries explicit tangent directions at every internal waypoint. These tangents serve directly as the endpoint derivatives $B'_i(0)$ and $B'_i(1)$ for the cubic Bézier segments of Section , eliminating the need for heuristic control-point placement. When λ is set to the robot’s minimum turning radius R , the curvature $\|B''(t)\|/\|B'(t)\|^3$ of each Bézier segment is bounded by $1/R$ along the entire smoothed trajectory, guaranteeing kinematic feasibility. Fraichard and Scheuer [Fraichard and Scheuer(2004)] provide an analogous continuous-curvature construction for clothoid-based paths; the Bézier formulation here is computationally simpler and integrates directly with the Dijkstra search.

Comparison with BFS. On uniform-cost grids, Dijkstra with a FIFO queue degenerates to BFS; the discrete solver path (Section) is thus a special case. The all-pairs SSSP phase runs once per instance, independent of k , and its output matrix d feeds all solver and simulation components.

Bézier Pipeline Benchmark

We evaluate the full Bézier pipeline—roadmap-based mTSP solving, C^1 Bézier smoothing with curvature-adaptive speed, and Dijkstra-weighted all-pairs distances—on 11 Moving AI benchmark maps with $n \in \{16, 32, 64, 128, 256, 512\}$ waypoints and $k \in \{4, 8, 16\}$ robots, totaling 558 runs. Roadmaps are generated by adaptive distance-transform sampling (Section) with $K = 8$ nearest-neighbor edge connectivity and `max_agents`=512. Maps `orz900d` and `maze-128-128-*` are excluded because their sparse geometry requires substantially higher node sampling rates to achieve a connected graph that retains all waypoints. The non-uniform n values in the results (e.g., $n = 486$ instead of $n = 512$) reflect waypoint filtering to the largest connected component of the roadmap; disconnected waypoints are dropped and the reported n is the retained count.

Table 7 reports win counts per solver across 175 (map, n, k) configurations where at least two solvers completed. Simulated annealing (SA) with Voronoi initialization and 2-OPT refinement dominates, achieving the best combined objective $f = M + \frac{1}{k} \sum C_r$ on 84.6% of all configurations.

Solver	Wins	Win %
SA	148	84.6%
k-means	21	12.0%
k-medoids	6	3.4%

Table 7: Bézier pipeline solver wins across 175 configurations on 11 maps.

SA’s dominance is most pronounced at $k = 4$ (57/62, 91.9%) and at $n = 64$ (33/33, 100% sweep). At larger $n \geq 256$, SA wins 20/27 configurations (74.1%), with k -means taking 4 and k -medoids 3. On maze-like maps (`ht_chantry`, `ht_mansion_n`), SA wins 31/36; on open

city maps (Berlin, Boston, Paris), SA wins 42/46, losing only to k -means at the largest n on Berlin ($n = 486$, $k = 16$). k -medoids wins only 6 configurations, all at $n \geq 256$, where its travel-balance penalties (§) become beneficial.

Table 8 shows representative scores for $k = 8$ at the highest available n per map.

Map	n	k-means	k-medoids	SA
Berlin_1_256	486	1506	2004	1742
Boston_0_256	507	1866	1979	1833
Paris_1_256	415	1429	1684	1608
brc202d	125	790	1161	810
den520d	454	1302	1840	1342
lak303d	121	341	516	368
ht_chantry	459	797	746	689
ht_mansion_n	405	788	1065	671
ost003d	512	1172	1187	964
w_woundedcoast	507	1863	2249	2040
warehouse-10-20-10-2-1	450	551	806	635

Table 8: Bézier pipeline scores ($k = 8$). Scores are $f = M + \frac{1}{k} \sum C_r$. Bold indicates best per row.

These results confirm that SA with Voronoi initialization is the most robust solver for the Bézier CMAPF pipeline, particularly on open maps and at moderate agent counts. The k -medoids clustering, while effective in the discrete setting (Section), underperforms on roadmaps because its balance penalties are calibrated for uniform grids and do not transfer well to non-uniform edge weights. The Dijkstra-Bézier pipeline produces trajectories that are both kinematically feasible (curvature-bounded by $a_{\max}$) and near-optimal with respect to the weighted roadmap graph.

Cross-Domain Comparison

Table 9 summarizes solver performance across the three CMAPF domains studied in this paper.

Domain	Maps	Configs	Best solver	Runner-up
Discrete (grid)	9	5	NRPA (60%)	SA (40%)
Continuous (roadmap)	18	203	NRPA (50%)	SA (38%)
Realistic (Bézier)	11	106	SA (56%)	NRPA (32%)

Table 9: Solver win rates on NRPA-participating configurations. In the discrete domain only $n \geq 16$ instances are counted (SA matches the exact optimum for $n \leq 14$).

Table 9 reports win rates on the subset of configurations where NRPA participated (112 runs in the realistic benchmark, allocated 100–500s per instance depending on map size). The key cross-domain trend is the inversion of the SA–NRPA ranking when curvature-adaptive Bézier speed replaces constant linear motion: NRPA leads 50% to 38% on continuous roadmaps but trails 32% to 56% in the realistic pipeline. This shift is explained by the non-uniform edge traversal times introduced by curvature adaptation: NRPA’s distance-based policy assumes uniform costs, whereas

SA’s Voronoi initialization and local search adapt naturally to the weighted objective. On the 66 (map, n, k) configurations shared between the two benchmarks, SA improves from 64% to 74% while NRPA drops from 32% to 11% (all-solver basis).

Across all three domains, SA emerges as the most robust solver—it matches the exact optimum on small discrete instances ($n \leq 14$), competes evenly with NRPA on constant-speed roadmaps, and dominates the realistic Bézier pipeline. k -medoids clustering underperforms in continuous and realistic settings (2–3%), despite being competitive in the discrete domain; its balance penalties are calibrated for uniform grids and do not transfer to non-uniform edge weights. k -means with 2-OPT provides a consistent baseline (9–11% wins) at sub-second runtime, making it suitable for rapid prototyping.

Conclusion

We studied Combinatorial Multi-Agent Path Finding in both discrete grid-based and continuous roadmap-based settings.

Discrete CMAPF

Three problem variants were evaluated: CMAPF, AMAPF, and DMAPF. For CMAPF and AMAPF, when only opposing edge and node conflicts occur on the overlay of assigned shortest paths, the collision resolution is optimal with respect to the underlying assignment. A free-cell memory optimization reduces BFS storage from $n \times W \times H$ to $n \times F$, enabling experiments at $n = 6,820$ that were previously impossible. Delta encoding achieves $330\times$ compression over naive output; binary frame encoding further reduces the full benchmark suite from 19 GB to under 1 GB. All node and edge collisions were resolved in every benchmark run.

Continuous CMAPF

The continuous variant replaces the grid with a roadmap graph, supporting three generation methods (randomized, skeleton, visibility). Robot paths are smoothed via cubic Bézier curves with C^1 continuity and constant-speed arc-length parameterization. Collisions are resolved by identity exchange at analytically precomputed intersection points, making the simulation deterministic and framerate-independent. The identity exchange is proven to be the physically correct resolution for elastic collisions of equal-mass point particles: conservation of momentum and kinetic energy force velocity exchange, equivalent to role exchange on the shared edge. For finite-radius robots, the collision region is bounded by a dilated interval of length $2r$ along the edge. For infinitesimally small robots, only edge collisions need be considered. Dijkstra’s algorithm computes exact all-pairs shortest paths on the weighted roadmap in $O(n|V| \log |V|)$ time, replacing BFS. If the mTSP is solved exactly on the graph, the continuous solution is optimal with respect to that graph. All analytically precomputed collisions were resolved in every

benchmark run. As with the discrete case, adaptive sampling enables the pipeline to handle all maps in the Moving AI benchmark set.

References

- [Boroujerdi and Uhlmann(1998)] Boroujerdi, A.; and Uhlmann, J. K. 1998. An Efficient Algorithm for Computing Least Cost Paths under Turn Angle Constraints. *Information Processing Letters*, 67(6): 317–321.
- [Dijkstra(1959)] Dijkstra, E. W. 1959. A Note on Two Problems in Connexion with Graphs. *Numerische Mathematik*, 1(1): 269–271.
- [Edelkamp and Plaku(2014)] Edelkamp, S.; and Plaku, E. 2014. Multi-Goal Motion Planning with Physics-Based Game Engines. In *Proceedings of the IEEE Conference on Computational Intelligence and Games (CIG)*, 115–122.
- [Farin(2002)] Farin, G. 2002. *Curves and Surfaces for CAD: A Practical Guide*. Morgan Kaufmann, 5th edition.
- [Fraichard and Scheuer(2004)] Fraichard, T.; and Scheuer, A. 2004. From Reeds and Shepp’s to Continuous-Curvature Paths. *IEEE Transactions on Robotics*, 20(6): 1025–1035.
- [Goldstein, Poole, and Saffo(2002)] Goldstein, H.; Poole, C. P.; and Saffo, J. L. 2002. *Classical Mechanics*. Addison-Wesley, 3rd edition.
- [Gravesen(1997)] Gravesen, J. 1997. Adaptive Subdivision and the Length and Energy of Bézier Curves. *Computational Geometry*, 8(1): 13–31.
- [Helsgaun(2017)] Helsgaun, K. 2017. An Extension of the Lin–Kernighan–Helsgaun TSP Solver for Constrained Traveling Salesman and Vehicle Routing Problems. Technical report, Roskilde University.
- [Jonker and Volgenant(1987)] Jonker, R.; and Volgenant, A. 1987. A Shortest Augmenting Path Algorithm for Dense and Sparse Linear Assignment Problems. *Computing*, 38(4): 325–340.
- [Le and Plaku(2017)] Le, D.; and Plaku, E. 2017. Cooperative Multi-Robot Sampling-Based Motion Planning with Dynamics. In *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, 187–195.
- [Le and Plaku(2018)] Le, D.; and Plaku, E. 2018. Cooperative, Dynamics-Based, and Abstraction-Guided Multi-Robot Motion Planning. *Journal of Artificial Intelligence Research (JAIR)*, 62: 721–752.
- [Rashidian, Plaku, and Edelkamp(2014)] Rashidian, S.; Plaku, E.; and Edelkamp, S. 2014. Motion Planning with Rigid-Body Dynamics for Generalized Traveling Salesman Tours. In *Proceedings of the ACM SIGGRAPH Conference on Motion in Games (MIG)*, 87–96.

- [Rashidian, Plaku, and Edelkamp(2015)] Rashidian, S.; Plaku, E.; and Edelkamp, S. 2015. Motion Planning with Rigid-Body Dynamics for Generalized Traveling Salesman Tours. *Computer Animation and Virtual Worlds*, 26(3–4): 217–226.
- [Sturtevant(2012)] Sturtevant, N. R. 2012. Benchmarks for Grid-Based Pathfinding. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(2): 144–148.